BCSCHOOL

Empowering Digital Skills For The Jobs Of The Future

by

BC Soft
consulenza sistemi informativi

AiRED
Artificial Intelligence Ricerca e Sviluppo

JAVA JEE
-
Operatori e Gestione del flusso di esecuzione

TEACHER

Vincenza Ruvolo

vincenza.ruvolo@bcsoft.net

Sommario

- Operatori di base
- Costrutti di programmazione semplici
- Costrutti di programmazione avanzati
- Introduzione ad UML

Operatori di base

Operatore d'assegnazione

L'operatore = permette di assegnare un valore ad una variabile.

```
int variabile1 = 1;
```

Operatori di base

La seguente tabella riassume gli operatori aritmetici semplici definiti dal linguaggio Java:

Descrizione	**Operatore**
Somma	+
Sottrazione	-
Divisione	/
Moltiplicazione	*
Modulo	%

Operatori di base

L’unico operatore che può risultare non familiare al lettore è l’operatore modulo(%). Il risultato dell’operazione modulo tra due numeri coincide con il resto della divisione fra essi.

Operatori di base

Ad esempio:

5 % 3 => 2

10 % 2 => 0

100 % 50 => 0

Operatori di base

Java ha ereditato dalla sintassi del linguaggio C anche altri operatori, sia binari (con due operandi) che unari (con un solo operando).

Operatori di base

Descrizione	Operatore
Somma ed assegnazione	+=
Sottrazione ed assegnazione	-=
Moltiplicazione ed assegnazione	*=
Divisione ed assegnazione	/=
Modulo ed assegnazione	%=

Operatori (unari) di pre e post-incremento (e decremento)

Descrizione	**Operatore**	**Esempio**
Pre-incremento di un'unità	`++`	`++i`
Pre-decremento di un'unità	`--`	`--i`
Post-incremento di un'unità	`++`	`i++`
Post-decremento di un'unità	`--`	`i--`

Operatori bitwise

La seguente tabella mostra tutti gli operatori bitwise (ovvero che eseguono operazioni direttamente sui bit) definiti in Java:

Descrizione	**Operatore**
NOT	~
AND	&
OR	|
XOR	^

Operatori bitwise

Shift a sinistra	<<
Shift a destra	>>
Shift a destra senza segno	>>>
AND e assegnazione	&=
OR e assegnazione	|=
XOR e assegnazione	^=
Shift a sinistra e assegnazione	<<=
Shift a destra e assegnazione	>>=
Shift a destra senza segno e assegnazione	>>>=

Operatori bitwise

Si precisi che l’operatore NOT (~) è un operatore unario, dato che si applica ad un solo operando.

Operatori bitwise

Gli operatori AND &, OR | e XOR ^, si applicano a coppie di operandi, e svolgono le relative operazioni logiche di conversioni di bit riassunte nella seguente tabella di verità:

Operando1	Operando2	Op1 AND Op2	Op1 OR Op2	Op1 XOR Op2
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

Operatori bitwise

Gli operatori di shift (operatori di scorrimento) provocano lo scorrimento di un certo numero di bit verso una determinata direzione. Il numero dei bit da scorrere è rappresentato dall’operando a destra dell’operazione.

Operatori di bitwise

La forma generale dell'istruzione di SHIFT A DESTRA è del tipo:

Variabile_intera >> numero_posizioni

Ciò significa che i bit di *variabile_intera* vengono tutti spostati a destra di *numero_posizioni.*

Operatori di bitwise

Esempio:

6 >> 2

$6_{10} = 0000110_2$

0 0 0 0 0 0 1 1 0

0 0 0 0 0 0 0 0 1

Per ogni bit che esce entra uno 0

6 >> 2 = 1

Operatori di bitwise

La forma generale dell'istruzione di SHIFT A SINISTRA è del tipo:

Variabile_intera << numero_posizioni

Ciò significa che i bit di *variabile_intera* vengono tutti spostati a sinistra di *numero_posizioni.*

Operatori di bitwise

Esempio:

6 << 2

$6_{10} = 0000110_2$

0 0 0 0 0 0 1 1 0

0 0 0 0 1 1 0 **0 0**

Per ogni bit che esce entra uno 0

6 << 2 = 24

Operatori relazionali o di confronto

Il risultato delle operazioni basate su operatori relazionali è sempre un valore boolean, ovvero true o false.

Operatore	**Simbolo**	**Applicabilità**
Uguale a	==	Tutti i tipi
Diverso da	!=	Tutti i tipi
Maggiore	>	Solo i tipi numerici
Minore	<	Solo i tipi numerici
Maggiore o uguale	>=	Solo i tipi numerici
Minore o uguale	<=	Solo i tipi numerici

Operatori logico - booleani

I seguenti operatori utilizzano solo operandi di tipo booleano, che vengono detti operatori logico – booleani e il risultato di un’operazione basata su tali operatori è sempre di tipo boolean.

Operatori logico - booleani

Descrizione	Operatore
NOT logico	!
AND logico	&
OR logico	|
XOR logico	^
Short circuit AND	&&
Short circuit OR	||

AND e assegnazione	&=
OR e assegnazione	|=
XOR e assegnazione	^=

Operatori logico - booleani

A differenza degli operatori bitwise, gli operandi a cui si applicano gli operatori booleani devono essere solo di tipo booleano.

Concatenazione di stringhe con +

In Java l'operatore +, oltre ad essere un operatore aritmetico, è anche un operatore per concatenare stringhe.

```
String nome = "James";
String cognome = "Gosling";
String nomeCompleto = "Mr. " + nome + " " + cognome;
```

Priorità degli operatori

Nella seguente tabella sono riportati, in ordine di priorità, tutti gli operatori di Java.

separatori	`.[]();,`
da sx a dx	`++ -- + - ~ ! (tipi di dati)`
da sx a dx	`* / %`
da sx a dx	`+ -`
da sx a dx	`<< >> >>>`
da sx a dx	`< > <= >= instanceof`
da sx a dx	`== !=`
da sx a dx	`&`

Priorità degli operatori

da sx a dx	^
da sx a dx	|
da sx a dx	&&
da sx a dx	||
da dx a sx	? :
da dx a sx	= *= /= %= += -= <<= >>= >>>= &= ^= |=

Costrutti di programmazione semplici

In ogni linguaggio di programmazione esistono costrutti che permettono al programmatore di controllare la sequenza di esecuzione delle istruzioni.

Costrutti di programmazione semplici

Si hanno due tipi di categorie:

- **Condizioni** (o strutture di controllo **decisionali)**: permettono, durante la fase di runtime, una scelta tra l'esecuzione di istruzioni diverse, a seconda che sia verificata o meno una specificata condizione.

Costrutti di programmazione semplici

- **Cicli** (o strutture di controllo **iterative**): consentono, in fase di runtime, di decidere il numero di esecuzioni di determinate istruzioni.

Il costrutto if

Il costrutto **if** è una condizione che permette di prendere semplici decisioni basate su **un'espressione booleana**. Un'espressione booleana è un'espressione che come risultato può restituire solo valori di tipo boolean, vale a dire **true o false.**

Il costrutto if

```
if (espressione-booleana){
    statement;
}
```

Esempio:

```
if (numeroLati == 3)
    System.out.println("Questo è un triangolo");
```

Il costrutto if

È possibile estendere la potenzialità del costrutto if mediante la parola chiave else:

```
if (espressione-booleana){
                    istruzione 1;
}
else{
                    istruzione 2;
}
```

Il costrutto if

Esempio:

```
if (numeroLati == 3)
   System.out.println("Questo è un triangolo");
else
   System.out.println("Questo non è un triangolo");
```

Il ragionamento da fare è il seguente:

```
Se il numero di lati è uguale a 3
   stampa "Questo è un triangolo"
altrimenti
   stampa "Questo non è un triangolo"
```

Il costrutto if

È possibile avere strutture più complesse utilizzando costrutti nidificati:

```
if (espressione-booleana) {
   if (variabile_1 == 0) {
      statement_1;
   } else {
      statement_2;
   }
   statement_3;
   ...;
   statement_k;
} else if (variabile_2 > 0){
    statement_k+1;
    ...;
   statement_n;
   } else {
     statement_z;
   }
}
```

Il costrutto while

Il ciclo **while** permette di eseguire più volte (iterare) uno statement, e ha la seguente sintassi:

```
[inizializzazione;]
while (espressione-booleana) {
        statement;
        [aggiornamento-iterazione;]
}
```

Il costrutto while

Da un punto di vista pratico, ad esempio, si avrà:

```
public class WhileDemo {
    public static void main(String args[]) {
        int i = 1;
        while (i <= 10) {
            System.out.println(i);
            i++;
        }
    }
}
```

Il costrutto while

Il ragionamento da fare è il seguente:

Inizializzo i a 1

Finché i non arriverà a 11
verrà stampata i
e verrà incrementato i di 1

Quando i varrà 11 uscirà dal ciclo

Il costrutto for

Il ciclo for, è probabilmente il ciclo più completo che offre il linguaggio.

Ciclo for ad uno statement:

```
for(inizializzazione; espressione booleana; aggiornamento){

        statement;

}
```

Il costrutto for

Ciclo for a più statement:

```
for(inizializzazione; espressione booleana; aggiornamento){

    statement_1;
    ...;
    statement_n;

}
```

Ciclo while VS Ciclo for

La sintassi del ciclo for è molto flessibile e compatta. Se nel while si utilizzano le parentesi tonde solo per l'espressione booleana, nel for vengono utilizzate rispettivamente prima l'inizializzazione di una variabile, poi l'espressione booleana ed infine l'aggiornamento che sarà eseguito ad ogni iterazione.

Ciclo while VS Ciclo for

```
for(inizializzazione; espressione booleana; aggiornamento){
        statement;
}


[inizializzazione;]
while (espressione-booleana) {
        statement;
        [aggiornamento-iterazione;]
}
```

Ciclo while

Il ciclo while è utilizzato **soprattutto quando non si sa quanto a lungo verranno eseguite le istruzioni** e soprattutto nei cicli infiniti, la sintassi è banale:

```
while (true) {
        . . .;
}
```

Il costrutto do

Il ciclo **do-while** è il ciclo meno utilizzato.

È fondamentale nel caso in cui si desideri la certezza che le istruzioni in un ciclo siano **eseguite almeno nella prima iterazione.**

Il costrutto do

```
[inizializzazione;]
do {
   statement;
   [aggiornamento iterazione;]
} while (espressione booleana);
```

In questo caso viene eseguito prima il blocco di codice e poi viene valutata l'espressione booleana (condizione di terminazione) che si trova a destra della parola chiave while. Se la condizione è verificata ricomincia il ciclo(codice del do) se è falsa si ferma.

Il costrutto switch

Il costrutto **switch** si presenta come alternativa al costrutto if. A differenza dell'if non è possibile utilizzarlo in ogni situazione in cui c'è bisogno di scegliere tra l'esecuzione di parti di codice diverse.

Il costrutto switch

```
switch (variabile di test)  {
   case valore_1:
      statement_1;
   break;
   case valore_2: {
      statement_2;
      statement_3;
   }
   break;
   case valore_3:
   case valore_4: { //blocco di codice opzionale
      statement_k+1;
      ...........;
   }
   break;
   default: {     //clausola default opzionale
      statement_j+1;
      ...........;
      statement_n;
   }
}
```

Il costrutto switch

Da un punto di vista pratico si consideri il seguente codice:

Il costrutto switch

```
public class SwitchStagione {
 public static void main(String args[]) {
  int mese = 4;
  String stagione;
  switch (mese) {
   case 12:
   case 1:
   case 2:
    stagione = "inverno";
   break;
   case 3:
   case 4:
   case 5:
    stagione = "primavera";
   break;//senza questo  break si ha estate
   case 6:
   case 7:
   case 8:
    stagione = "estate";
   break;
   case 9:
   case 10:
   case 11:
    stagione = "autunno";
   break;
   //la clausola default è opzionale
   default:
    stagione = "non identificabile";
  }
  System.out.println("La stagione è " + stagione);
 }
}
```

L’operatore ternario

L’operatore ternario (detto anche operatore condizionale), simile al costrutto if, può regolare il flusso di esecuzione come una condizione.

L’operatore ternario

La sintassi è:

variabile = (espressione_booleana) **?** espr1 **:** espr2;

L’operatore ternario

se il valore dell’espressione_booleana è true

si assegna a variabile il valore di espr1

Altrimenti

si assegna a variabile il valore di espr2

Requisito indispensabile è che il tipo della variabile e quello restituito da espr1 ed espr2 siano compatibili.

break e continue

La parola chiave **break** è un comando capace di fare terminare il costrutto switch.

Ma break è utilizzabile anche per far terminare un qualsiasi ciclo.

break e continue

```
int i = 0;
while (true) { //ciclo infinito
    if (i > 10) {
        break;
    }
    System.out.println(i);
    i++;
}
```

break e continue

La parola chiave **continue** fa terminare non l’intero ciclo, ma solo l’iterazione corrente (ovvero fa saltare alla prossima iterazione).

break e continue

```
int i = 0;
do {
    i++;
    if (i == 5) {
        continue;
    }
    System.out.println(i);
} while(i <= 10);
```

break e continue

break e **continue** possono utilizzare etichette (label) per specificare, solo nel caso di cicli annidati, su quale ciclo devono essere applicati. Ad esempio:

break e continue

```
int j = 1;

pippo: //possiamo dare un qualsiasi nome ad una label

while (true) {
   while (true) {
      if (j > 10)
         break pippo;
      System.out.println(j);
      j++;
  }
}
```

break e continue

Una **label** ha quindi la seguente sintassi:

```
nomeLabel:
```

break e continue

Una label può essere posizionata solo prima di un blocco di codice che di solito è definito da un ciclo.

Gli array in Java

Un array è una collezione indicizzata di dati primitivi, o di reference o di altri array. Permettono di utilizzare un solo nome per individuare una collezione costituita da vari elementi, che saranno accessibili tramite indici interi.

Gli array in Java

In Java gli array sono, in quanto collezioni, oggetti.

Per utilizzare un array bisogna passare attraverso tre fasi:

- dichiarazione
- creazione
- inizializzazione.

Gli array in Java: dichiarazione

Si consideri il seguente **snippet:**

```
char alfabeto[];
```

oppure

```
char[] alfabeto;
```

```
File files [];
```

oppure

```
File[] files;
```

Gli array in Java: dichiarazione

Nella prima riga dichiariamo un array di char (tipo primitivo), nella seconda dichiariamo un array di istanze di File (classe appartenente al package java.io)

Gli array in Java: dichiarazione

Quindi per dichiarare un array, basta posporre (oppure anteporre) una coppia di parentesi quadre all'identificatore.

Gli array in Java: creazione

È obbligatorio specificare al momento dell’istanza dell’array la dimensione dell’array stesso.

Gli array in Java: creazione

La sintassi è la seguente:

```
char[] alfabeto = new char[21];

File[] files = new File[3];
```

Gli array in Java: Inizializzazione

Per inizializzare un array, bisogna inizializzarne ogni elemento singolarmente.

Ad esempio:

```
alfabeto [0] = 'a';
alfabeto [1] = 'b';
alfabeto [2] = 'c';
alfabeto [3] = ‘d’;
. . .
alfabeto [20] = 'z';
```

Gli array in Java: Inizializzazione

Ad esempio:

```
files [0] = new File("HelloWorld.java");
files [1] = new File("HelloWorld.class");
files [2] = new File("HelloWorld.html");
```

Gli array in Java: Inizializzazione

L’indice di un array inizia sempre da zero.

Un array dichiarato di 21 posti, avrà come indice minimo 0 e massimo 20 (un array di dimensione n implica il massimo indice a n-1).

Gli array in Java: Inizializzazione

È possibile effettuare in Java i tre passi contemporaneamente tramite una particolare sintassi breve

```
char alfabeto [] = {'a', 'b', 'c', 'd', 'e', 'z'};
File files [] = {new File("HelloWorld.java"),
                 new File("HelloWorld.class"),
                 new File("HelloWorld.html")};
```

Gli array in Java: Inizializzazione

Si noti la differenza tra un array dichiarato di tipo di dato primitivo o complesso. Il primo contiene direttamente i suoi elementi. Il secondo contiene solo reference, non gli elementi stessi.

Gli array in Java

La variabile chiamata length restituisce la dimensione effettiva dell'array stesso.

Quindi: alfabeto.length → 21

Array Multidimensionali

Esistono anche array multidimensionali che sono array di array.

Array Multidimensionali

String [] args in Java è una matrice di stringhe che memorizza gli argomenti passati dalla riga di comando durante l'avvio di un programma. Tutti gli argomenti della riga di comando sono memorizzati in quell'array.

Laboratorio

Scrivere un programma in Java che memorizza in un array 5 elementi interi e ne stampa il contenuto.

Laboratorio: Soluzione

```
package esercizioEsempio;

public class LeggiScrivi {

	public static void stampa(int[] v){
		System.out.println("array letto: ");
		for (int i=0; i<v.length; i++)
		   System.out.print(v[i] + " ");
		System.out.println();
	}

	public static void main(String[] args) {
		int[] array = new int[5];

		array[0] = 4; array[1] = 7; array[2] = 20; array[3] = 9; array[4] = 10;

		stampa(array);
	}
}
```

Domande e approfondimenti